

A Report On The Transformer

Koppany Dodony

October 2023

“You become what you give your attention to”

-Seneca

Contents

1	Attention	3
1.1	Clarifying The Dimensions of a Single Attention-head	3
1.2	Dimensions of Q,K,V in a single head	4
1.3	Conclusion	5
1.4	Multi-Head Attention	6
1.5	dimensions of the projected Q,K,V	7
1.6	The case a column vectors	8
1.7	The origin of Q,K,V	9
1.8	Different variations of attention	10
2	From the Input Data to The Input of Attention	11
2.1	GPT Tokenizer	11
2.2	Initial inputs	11
2.3	Tokenization	11
2.4	Embedding	11
2.5	Positional Encoding	12
2.6	The Fixed Version of Positional encoding	12
3	Position Wise Feed Forward	13
4	Architecture of The Model	13
4.1	Analysis of The Encoder	14
4.2	Analysis of The Decoder	14
5	Batching and Regularization	15
5.1	Batches	15
5.2	Regularization	15
6	Training	15
6.1	Training in General	15
6.2	The Adam Optimizer	16
6.3	Loss Functions Of Transformer	16
7	Appendix	17
7.1	The Architecture of the Three Different Attention Mechanisms	17
8	References	17

Introduction

Most of this work follows the paper Attention is All You Need [2] while we are aiming to offer a proper interpretation of the paper, we encourage the reader to read that article independently from this work. While we have tried to be as punctual and precise as we can, we maintain the right for errors and we also have the honest hope to reach someone who can help the improvement of this work.

The aim of this work is an introduction to the transformer. In the first three chapters we introduce the building blocks of the model and then we give a brief introduction to it's architecture. In chapter 5 we add regularization for better runtime and in the final chapter we'll see an application of the model. We will try to give a brief introduction to the model's mathematical background and some figures also for better understanding in the appendix.

1 Attention

We have found that clarifying the different sizes and dimensions of the model is a cardinal step regarding the correct implementation of the model. We'd like to share what we could find out based on the paper. Our aim is to prove each of the steps by citation where we make an implication.

1.1 Clarifying The Dimensions of a Single Attention-head

We begin our journey by assuming that we have the Q, K, V matrices given in the paper. our first claim is that they really are at least rank 2 matrices.

"An attention function can be described as mapping a query and a set of key-value pairs to an output, where the query, keys, values, and output are all vectors."

This describes what an attention function should do it is 3.2's first sentence. what we conclude from here that

$$Attention(\underline{q}, K, V) = \underline{result}$$

, where

- $\underline{q}$ and $\underline{result}$ is a vector.
- K, V are matrices since they are sets of key-value pairs, where keys, values are vectors, so we imply that they are stacked vectors.
- Since K and V are representing a set of key-value pairs they must store the same amount of vectors.

"In practice, we compute the attention function on a set of queries simultaneously, packed together into a matrix Q . The keys and values are also packed together into matrices K and V . We compute the matrix of outputs as: $Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) V$ "

d_k is a constant and it is the length of a key vector (3.2.1 first sentence).

I think that this proves that Q, K, V and the output are at least rank 2 matrices, as mentioned in the citation.

Next we would like to find out the dimensions of those matrices. There are a couple of things to consider before further investigation: By formula (1) we can assume that:

1. We must be able to multiply Q and K^T .
2. An Attention function maps a $(\underline{q}, K, V)$ triple to a vector. So it is reasonable to think that in practice we compute Attention for each $\underline{q}$ that stacked together in matrix Q and the results (one result is a vector) are also stacked together in a matrix.
3. We must be able to multiply $\text{softmax}(QK^T)$ and V , softmax does not reduce or increase dimension so we can assume that it is possible to multiply QK^T and V .

1.2 Dimensions of Q,K,V in a single head

We are assuming for now that we do not use multiheads yet.

From 3.1's last paragraph:

"That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself. To facilitate these residual connections, all sub-layers in the model, as well as the embedding layers, produce outputs of dimension $d_{\text{model}} = 512$."

So before Attention and after Attention we have d_{Model} long vectors, we still do not know how many. From 3.2.1's first sentence:

"We call our particular attention "Scaled Dot-Product Attention" (Figure 2). The input consists of queries and keys of dimension d_k , and values of dimension d_v ." from 3.2.2's last sentence: "In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $d_k = d_v = d_{\text{model}} = h = 64$. Due to the reduced dimension of each head, the total computational cost is similar to that of single-head attention with full dimensionality."

Putting it together:

1. d_k and d_v are the lengths of the input vectors after applying multihead attention, at first we would like to present a single attention head. Since the purpose of those lengths is to partition the inputvectors we can assume that we skip that step for now, our next citation shows that they are equal in the model so we have nothing to loose if we set this value to d_{Model} and apply the algorithm to one Attention head.
2. Q, K and V are having the same amount of vectors, since $d_k = d_v$ so it must be number of columns of the given matrices.
3. d_{Model} is a common dimension of Q, K, V and it is the amount of columns of the Q, K, V matrices.

We should be back to the question that how many such inputvectors we have before Attention? For this we have to go to the tokenization process we make the following two claims:

1. The output of tokenization is a matrix with $|\text{NumberOfTokens}|$ rows and d_{Model} columns.
2. Positional encoding has no effect on the size of the output mentioned above.

This claim 1 is based on 3.4:

"Similarly to other sequence transduction models, we use learned embeddings to convert the input tokens and output tokens to vectors of dimension d_{model} ."

What implies that we have a corresponding d_{Model} long vector to each output token.

And claim 2 and claim 1:

"The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed"

So we conclude that the input of the Attention is a matrix with $|NumberOfTokens|$ rows and d_{Model} columns, since positional encoding has no effect on the size of the output.

1.3 Conclusion

The list below shows claims that we could directly conclude from the paper, assuming we have only one attention head.

1. the input of the Attention is a matrix with d_{Model} columns and $|NumberOfTokens|$ rows.
2. the output of an Attention head has the same size as the input, since we must be able to add them together.
3. Q, K and V stores the same amount of vectors.
4. d_{Model} is the number of columns of Q, K, V .
5. We must be able to multiply Q and K^T .
6. We must be able to multiply $softmax(QK^T)$ and V

From the claims above and basic rules of matrix multiplication we can construct the following list of implications:

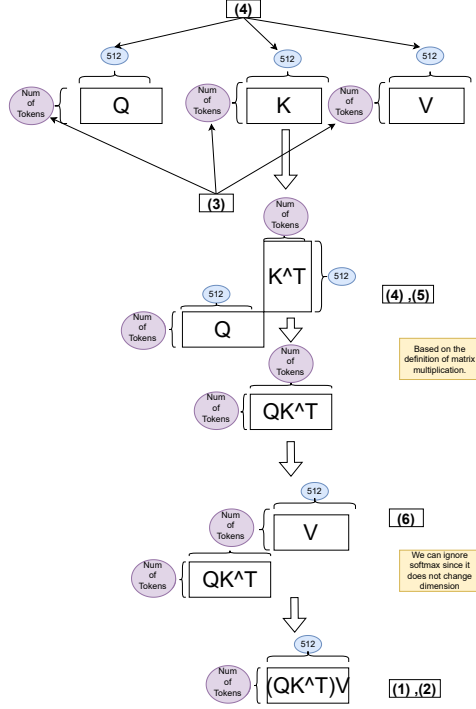


Figure 1: Dimensions of the Q,K,V matrices

It was quite a long and dry process and it lead to really small amount of information, since the conclusion of the whole can be summarised in the following sentence: $Q, K, V \in \mathbb{R}^{|NumberofTokens| \times |d_{Model}|}$ in case of a single head attention.

An important note here, that the paper does not discuss if those so called "vectors" are row, or column vectors. It is common in the machine learning literature to refer to them as columns vectors. In this process we have referred to them as row vectors, it is left to prove that the paper also means a row vector when it uses the word vector. The reason behind this will be clear assuming our conclusion we've made and using rules of algebra. After the discussion of the next section we will play the scenario when "vectors" are column vectors and identify the problems comming from that interpretation.

1.4 Multi-Head Attention

Now our purpose is to investigate the multi-head attention mechanism and understand it's function and architecture. To do so, we start with some citations from the paper. For the following claims we cite from 3.2.2:

"Instead of performing a single attention function with d_{model} -dimensional keys, values and queries, we found it beneficial to linearly project the queries, keys and values h times with different, learned linear projections to d_k , d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel, yielding d_v -dimensional output values. These are concatenated and once again projected, resulting in the final values, as

depicted in Figure ...

$$Multihead(Q, K, V) = Concat(head_1 \dots head_h)W^O$$

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

Where the projections are parameter matrices $W_i^Q \in \mathbb{R}^{d_{model} \times d_k}$, $W_i^K, W_i^V \in \mathbb{R}^{d_{model} \times d_v}$ and $W^O \in \mathbb{R}^{hd_v \times d_{model}}$... In this work we employ $h = 8$ parallel attention layers, or heads. For each of these we use $dk = dv = d_{model} = h = 64$.

So what happens here is almost the same we've seen before. Instead of applying attention to the whole input, that had the size $|NumberOfTokens| \times d_{Model}$ We project the input linearly into $|NumberOfheads|$ layers, and perform the Attention function on those projections. The projection matrices are learned and after the projections we concatenate the results and apply another linear projection to the concatenated output of Attention.

The paper uses the notation of d_k and d_v when it comes to the number of vectors we put into the mentioned matrices, due to the citation we can ignore it from now and use the same dimension $d := \frac{d_{Model}}{|NumberOfHeads|}$ for each.

Note 1. The reason behind the notation comes from the encoder-decoder attention, where the sizes of the $Q_{decoder}$ and $K_{encoder}, V_{encoder}$ may differ. We ask the reader to ignore this for now and come back after the understanding of the encoder.

1. We have $Q, K, V \in \mathbb{R}^{|NumberOfTokens| \times d_{Model}}$ before projection.
2. We have a given parameter of the model called *Number of heads*.
3. We apply $|NumberOfheads|$ linear projections of Q, K, V and we get

$$QW_i^Q, KW_i^K, VW_i^V$$

matrices where the W_i^Q, W_i^K, W_i^V matrices are the i th head's projection matrices correspond to Q, K, V respectively.

4. We apply an Attention function to each head:

$$Attention(QW_i^Q, KW_i^K, VW_i^V) \quad \forall 1 \leq i \leq 8, i \in \mathbb{N}.$$

5. We concatenate the results and apply another linear projection.
6. Finally we apply residual connections i.e. we add our or original input to the result we've got in the last step and apply a softmax function after this.

1.5 dimensions of the projected Q,K,V

We are again in the same situation and have no conclusion about the dimensions of the projected Q, K, V . Before we investigate this we make the following claim based on the last sections' claims. This is the time where we need to take all the results we had investigated before into account, so we

suggest the reader to ensure the claim we make in the following is consistent with our previously made ones.

- $Q, K, V \in \mathbb{R}^{|NumberOfTokens| \times d_{model}}$ and $W_i^Q, W_i^K, W_i^V \in \mathbb{R}^{d_{model} \times d}$ where $d = d_k = d_v$ and i is the i th head.

The result of the projection is indeed a matrix with $|NumberOfTokens|$ rows and d columns for QW_i^Q, KW_i^K and VW_i^V by rules of matrix multiplication. We have performed the projection $|NumberOfheads|$ times and we concatenate the results that is going to be a matrix with the size

$$|NumberOfTokens| \times d_{model}$$

since $d \cdot |NumberOfheads| = d_{model}$.

So all the projected versions of Q, K, V has the common size $|NumberOfTokens| \times d$.

Now let us assume we have each head and each projected version of the Q, K, V matrices. From here we have nothing else to do, just apply the process we have seen in the sections we've discussed the single head attention to each, concatenate the results and apply a Linear projection to the concatenated version. In the following flowchart on the next page one may see how the process happens where the inputs are the Q, K, V matrices and the output is the concatenated version of the multi-head attention's output, we use no softmax now, since we are in chase of the correct dimensions at the moment. Besides providing the architecture in detail on the next page we also share a [link for implementation](#) to the reader.

1.6 The case a column vectors

One may wonder why we ave used row vectors while in the machine learning community it is common to refer to a vector as a column vector. The reason is that the authors of the paper did use the word vector in this unorthodox term. Let us assume that the paper refer to vectors as column vectors. The problem with this approach arises when we try to apply the Multi-head attention function and apply the linear projection. We show this for the Q matrix only other cases are identical to this. If $Q \in \mathbb{R}^{d_{model} \times |NumberOfTokens|}$ (so it consists of concatenated column vectors) the product QW_i^Q is only defined in that special case when $|NumberOfTokens| = d_{model}$, since $W_i^Q \in \mathbb{R}^{d_{model} \times d}$. In every other case it is not possible to make the projection by rules of matrix multiplication, so we can conclude that this approach makes no sense.

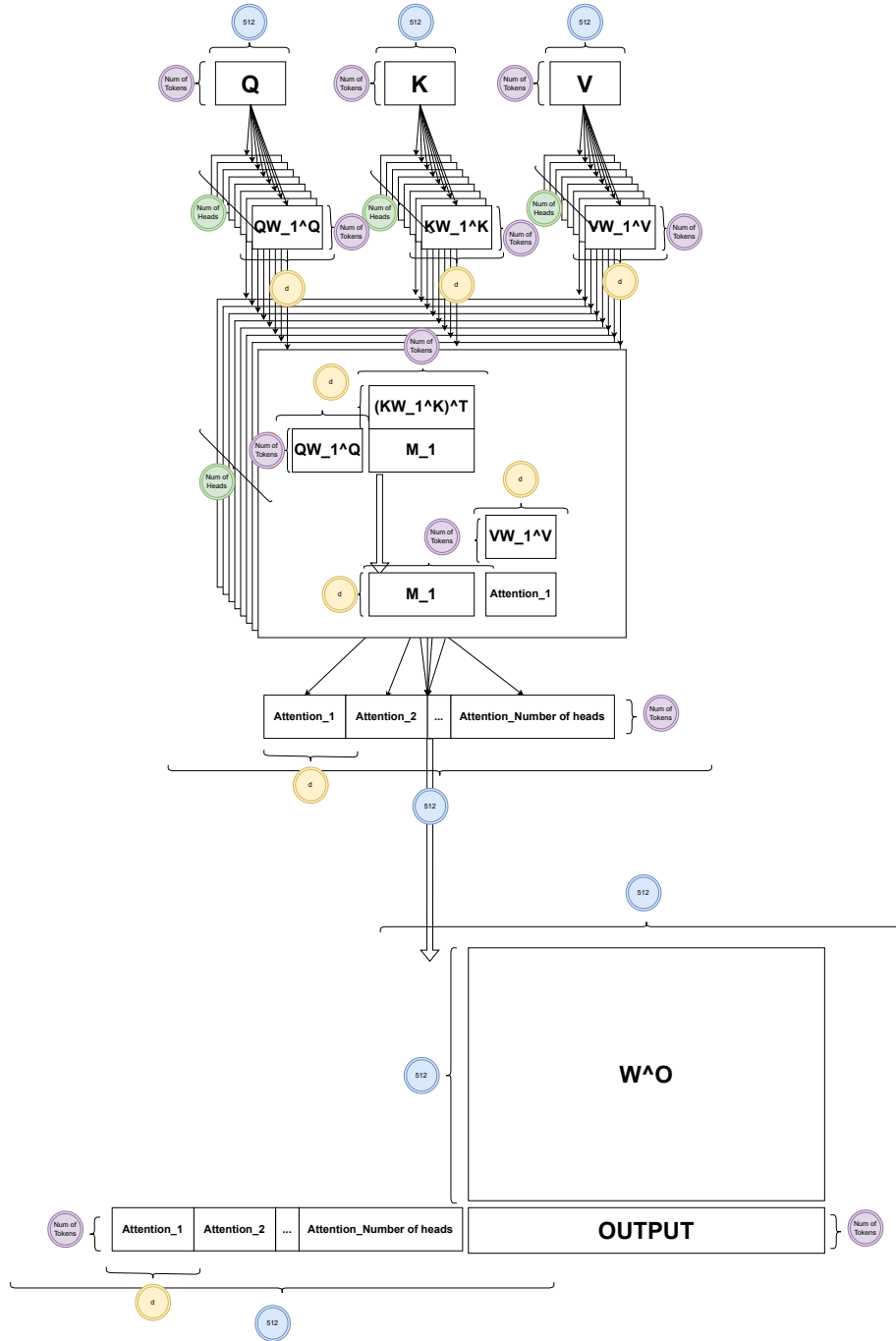


Figure 2: The multi-head attention mechanism

1.7 The origin of Q, K, V

In the previous sections we could see how the attention mechanism works in general, but we have assumed that the Q, K, V matrices are given inputs. We have investigated the size of the algorithm's input and also the sizes of Q, K and V , the question of how we get those matrices from the output

of the Positional encoding arises naturally. Unfortunately I have found no clue about this question in the article, neither with the help of GPT4 nor with AI forums on the internet. A lecture given by Lucas Beyer from Google Brain answers this. In his lecture on the transformer, he says those matrices are projected linearly from the output of positional encoding. It is in the [linked video](#) at 30:10. I also have e-mailed the authors regarding the question, since it is clearly not mentioned in the paper. I am still waiting for their responses and I will consider myself happy if I get any :). From this point we have to limit ourselves to the lecture given by Lucas Beyer and assume that those matrices are linearly projected versions of the output of the positional encoding in the first encoder layer. I will update the document if we get another clue on this topic. **update:** Llion Jones answered my e-mail and told me to follow a path what I guess is intuitive, and confirmed that we may apply linear projection. So we may add the projection of the output of positional encoding into those Q, K, V matrices as step 0 and if we do so, we can match the size of the input and output of the attention mechanism.

1.8 Different variations of attention

While the paper offers three different, yet very similar attention mechanisms we think the most cardinal and important part of our investigation had been reached. In the following we introduce those three mechanisms.

From 3.2.3:

"The Transformer uses multi-head attention in three different ways:

- In "encoder-decoder attention" layers, the queries come from the previous decoder layer, and the memory keys and values come from the output of the encoder. This allows every position in the decoder to attend over all positions in the input sequence. This mimics the typical encoder-decoder attention mechanisms in sequence-to-sequence models such as [31, 2, 8].
- The encoder contains self-attention layers. In a self-attention layer all of the keys, values and queries come from the same place, in this case, the output of the previous layer in the encoder. Each position in the encoder can attend to all positions in the previous layer of the encoder.
- Similarly, self-attention layers in the decoder allow each position in the decoder to attend to all positions in the decoder up to and including that position. We need to prevent leftward information flow in the decoder to preserve the auto-regressive property. We implement this inside of scaled dot-product attention by masking out (setting to $-\infty$) all values in the input of the softmax which correspond to illegal connections."

From this we can identify 3 different attention mechanisms:

1. **encoder's self-attention** is the attention had been shown in the previous chapter.
2. **decoder's self-attention** is almost the same as we discussed before, the only difference here is that we mask the values in the input of the softmax, that is the Q, K dot product and that means we keep the original Q, K dotproduct and set it's upper triangle part to $-\infty$.
3. **encoder-decoder attention** is again almost the same as the encoder's self-attention, the only difference here is that we keep the encoders's Q, K and the decoder's V and apply the attention mechanism with those Q, K, V .

Here we note that the reader can see why we needed that d_k and d_v parameter. It is clear that the input of size of the encoder and the tokenised input of the decoder may differ, while

From our claims above we construct the algorithms of the different Attention mechanisms, we encourage the reader to check their flowcharts in the appendix and the corresponding [code on this link](#).

2 From the Input Data to The Input of Attention

In the previous chapter we have investigated the output of the Positional encoding. In this chapter we are going to observe how can we transform our input data to tokens, embed them while mapping each input to a unique position.

2.1 GPT Tokenizer

The input of a tokenizer is a string. We transform the input into integers in some fixed vocabulary and make a look up into lookup table of vectors and feed them into the model.

We are going to use the byte-pair encoding algorithm: The main idea of the algorithm is looking for the most frequent character pairs in the corpus and replace them with a single character called token. So the algorithm can compress our input on the price of creating a larger vocabulary.

2.2 Initial inputs

In the model our input is a set of sentences and their translations. We can think about them as pairs where one of the pairs is a sentence in language A , and the other is its translation to language B .

2.3 Tokenization

Our first mission is to convert our input string into a data structure that we can use to process our text. One may find huge amount of papers on that topic, we start by citing the paper and look for the corresponding literature.

We call a mapping tokenization if its domain is a string, and it's range is a numerical vector. There are numerous tokenization algorithms, this paper mentions the byte-pair encoding [1] and word piece representation [3]. We let the reader follow the mentioned articles in search for the various tokenization processes. We continue are progression and assume that our data is already tokenized.

2.4 Embedding

The word embedding is referring to a conversion from tokens to vectors that represents our data in a continuous form. Our main goal now is to transform our tokens to d_{model} long float vectors, we've got used to. One may think about embedding as a linear projection. Our inputs are tokens and our outputs are the mentiond float vectors. One may find the corresponding [code on this link](#) .

2.5 Positional Encoding

We may start by the corresponding citation and conclude what we can. From 3.5:

Since our model contains no recurrence and no convolution, in order for the model to make use of the order of the sequence, we must inject some information about the relative or absolute position of the tokens in the sequence. To this end, **we add "positional encodings" to the input embeddings at the bottoms of the encoder and decoder stacks. The positional encodings have the same dimension d_{model} as the embeddings, so that the two can be summed.** There are many choices of positional encodings, learned and fixed. In this work, we use sine and cosine functions of different frequencies:

$$PE(pos, 2i) = \sin(pos/10000^{2i/d_{model}})$$

$$PE(pos, 2i + 1) = \cos(pos/10000^{2i/d_{model}})$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid. The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$. **We chose this function because we hypothesized it would allow the model to easily learn to attend by relative positions**, since for any fixed offset k, $PE(pos + k)$ can be represented as a linear function of $PE(pos)$. We also experimented with using learned positional embeddings [8] instead, and found that the two versions produced nearly identical results (see Table 3 row (E)). We chose the sinusoidal version because it may allow the model to extrapolate to sequence lengths longer than the ones encountered during training.

According to this citation we need to find a way to encode the positions of the tokens, that makes sense if we would like the model to keep the cohesion of the sentences and words followed by one another. Based on this citation we may choose the way of fixed positional encoding or a learned version of it. Since the fixed version is shared here in more detail we progress with that one.

2.6 The Fixed Version of Positional encoding

Assume we have Our embedded vectors, more precisely right now we have $|NumberOfTokens|$ vectors, each with d_{model} length. It is important that at this point, the order of those vectors align with the order of the tokens we've got from the input text. Let us assume, that $A \in \mathbb{R}^{|NumberOfTokens| \times d_{model}}$. By the above citation to encode positions we only have to construct a matrix $P \in \mathbb{R}^{|NumberOfTokens| \times d_{model}}$, where

$$p_{i,j} = \begin{cases} \sin\left(\frac{i}{10000^{\frac{2j}{d_{model}}}}\right) & \text{if } 2|j, \\ \cos\left(\frac{i}{10000^{\frac{2j}{d_{model}}}}\right) & \text{otherwise.} \end{cases}$$

After the construction of P we only need to add it to A , so

$$Output = A + P.$$

The reader can find the corresponding [code on this link](#).

3 Position Wise Feed Forward

In the following chapter we will introduce our last cardinal step that let us build up the transformer's architecture. It is called the Position wise feed forward layer. Its input is the output of an attention mechanism. First we start with the corresponding citation, make our claims and conclusions, afterwards we present the algorithm and also share the corresponding code. From 3.3

In addition to attention sub-layers, each of the layers in our encoder and decoder contains a fully connected feed-forward network, which is applied to each position separately and identically. This consists of two linear transformations with a ReLU activation in between.

$$FFN(x) = \max(0; xW1 + b1)W2 + b2$$

While the linear transformations are the same across different positions, they use different parameters from layer to layer. Another way of describing this is as two convolutions with kernel size 1. The dimensionality of input and output is $d_{model} = 512$, and the inner-layer has dimensionality $d_{ff} = 2048$.

Based on the citation we make the following claims:

- We indeed use feed forward after each attention mechanism.
- Applying FFN for each position means that we run $|NumberOfTokens|$ separate FFN for each input vectors, that encoding a token.
- The algorithm itself applies a linear projection to our inputvector and increase it's dimension to 2048,we apply a ReLU then we project it back to its original size with another linear projection. We use bias in each projection based on the formula.

From 3.1:

"... We employ a residual connection [10] around each of the two sub-layers, followed by layer normalization..."

So we apply residual connections and layer normalization also as we've seen before. The reader can view the corresponding [code on this link](#).

4 Architecture of The Model

The cardinal elements of the transformer had been shown in the previous chapters. Now our aim is to build up the whole model and at this moment our current knowledge fortunately let us do it. We have attached a picture of the architecture in figure 3.

As the reader can see on the figure all we need to do is put together what we already know. The whole model is called a transformer. The layers in the left-hand-side's gray box is called an encoder layer and at the right-hand-side it's called a decoder layer. We have a parallel flow of data: The bottom left-hand-side Input is our input language and the bottom right hand side Output is our target language. It is important to note that this "output" is only formally identical to the output of our model. The reader can see our "real output" in the top right corner. So let us introduce how the

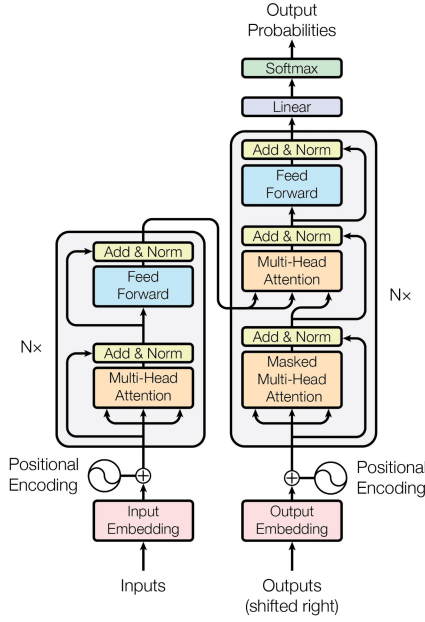


Figure 3: The model's architecture

information flows. We draw the reader's attention to the " Nx " can be seen next to the encoder and decoder layer. This Nx means that we run each of the encoder and decoder layers N times.

In the following we analyse the architecture in the direction of the information's flow. First we start with the encoder and then we switch to the decoder.

4.1 Analysis of The Encoder

- i; Assuming we have the input tokens we use our embedding process.
- ii; Afterwards we apply positional encoding.
- iii; The next step is the encoder's self Attention
- iv; Followed by a position-wise feed-forward Neural Network.
- v; we iterate step *iii*; and *iv*; N times, so N refers to the number of encoder layers.
- vi; We get our final output, that going to be the input of each decoder layer.

4.2 Analysis of The Decoder

Before we jump straight into the encoder we mention that each decoder layer has 2 inputs. One is coming from the target tokens and another comes from the encoder's output.

- i; Assuming we have the target tokens we apply our embedding method just as we did at the encoder.
- ii; Afterwards we apply positional encoding it is still identical to the encoder.

iii; Next we apply masked attention.

iv; Now we apply cross attention

v; After that we apply the position wise Feed Forward Neural Network

vi; We iterate the steps *iii*; *iv*; *v*; N times.

vii; for the last output in the chain we apply a Linear projection and a softmax function.

The reader can find the corresponding [code on this link](#)

5 Batching and Regularization

5.1 Batches

For better understanding we considered that our model processes the whole database in one step. The variable "*number_of_tokens*" referred to our tokenized input, where the input was the whole database. In practice this approach is really time consuming and it is also possible, that our hardware is not good enough for processing the whole database. That's why it is common in machine learning literature to divide the input database into a partition, that contains smaller chunks of the data, called batches. The reader may notice that it makes no difference if we apply the algorithm above to batches, the only thing we need to do before applying our model is partition our data into batches.

5.2 Regularization

For better runtime we may apply dropout, the article is quite straightforward on the topic of regularization, so we only cite, the corresponding part of the paper:

Residual Dropout We apply dropout [27] to the output of each sub-layer, before it is added to the sub-layer input and normalized. In addition, we apply dropout to the sums of the embeddings and the positional encodings in both the encoder and decoder stacks. For the base model, we use a rate of $P_{drop} = 0.1$.

Label Smoothing During training, we employed label smoothing of value $ls = 0.1$ [30]. This hurts perplexity, as the model learns to be more unsure, but improves accuracy and BLEU score.

6 Training

We give a really brief sketch about training in general, followed by the model's loss function and show the actual training loop.

6.1 Training in General

Assume we have an M model for a problem with input data $\underline{x}$ and parameters $\underline{\theta}$ and we also have an output: $M(\underline{x}, \underline{\theta})$. We would like to get some information out of the model. To do it we have to define a function $f(\underline{\theta}|\underline{x})$ called loss function that measures our success. This notation is referring to the fact

that our database is given and the function only depend on the values of $\underline{\theta}$. The input database is a subset of $Dom(f)$. The larger the loss functions values the further we are from our success. To maximize our success we have to find a good approximation of $\min f(M(\underline{x}, \underline{\theta}))$. Unfortunately it is not that easy a straightforward most of the time to find the minimum, we can get to saddle-points or find local (and not global) minimum. The optimization algorithms, that help us to find the minimum are usually iterative and require a starting vector. It is common to calculate the gradient of the loss-function and take steps on the domain in the direction of the gradient (or in the opposite direction). A common trick that (may) resolves the problems mentioned above is trying different starting points, but there is no general rule we could apply, there are kind of special manifolds where one trick works and others where they don't. We encourage the reader to study multi-variable analysis and geometry of surfaces and manifolds to dive deeper in this topic. An important note is if our loss function is differentiable and convex and bounded we always have the minimum. Note that if we miss any criteria mentioned above we may not be able to find the minimum with differentiation. Next we briefly talk about calculus, we encourage the reader to skip the reminder of this subsection if one familiar with gradients.

The most usual way to find our model's optimal parameters with respect to our loss function f is differentiating $f(M(\underline{x}, \underline{\theta}))$ with respect to $\underline{\theta}$. In most cases (i.e. When all the partial derivatives of f are continuous) the derivative of a function at $\underline{\theta}$ is equal to the gradient in $\underline{\theta} \in \mathbb{R}^n$, noted by $\nabla f(\underline{\theta})$, where

$$\nabla f(M(\underline{x}, \underline{\theta})) = \left(\frac{\partial M(\underline{x}, \underline{\theta})}{\partial \theta_1}, \frac{\partial M(\underline{x}, \underline{\theta})}{\partial \theta_2}, \dots, \frac{\partial M(\underline{x}, \underline{\theta})}{\partial \theta_n} \right).$$

In this sense the most important thing about our gradient, that it is a **vector** that show the **direction** of the highest possible increment of the function f in a small neighborhood of $\underline{x}$. A naive approach is to pick a point, calculate the gradient there and (if we would like to find maximum) make a **step** in the direction of the gradient and evaluate the function there and take the gradient there again, and make another step and so on. The literature refers to the length of the step as **learning rate**. Iterating this process we can always move in the direction of the highest increment so sooner or later we get to the maximum. It not always work, for example think about functions has no maximum. There are countless other examples, we encourage the reader to read some calculus for better understanding.

6.2 The Adam Optimizer

Now days Adam seems to be the most widely used optimizer in the AI community. The reason behind this is based on it's precision and fast convergence. The algorithm's basis is the gradient descent algorithm. Note that using Gradient descent requires the user to fine tune the learning rate and one may have to experiment with it: high learning rate can result in no convergence, while low learning rate cost us much time. Adam resolves this problem and introduces a changing learning rate in the following we introduce the algorithm and the theory behind it.

6.3 Loss Functions Of Transformer

We can apply various loss functions for various tasks. Now we consider translation so it seems beneficial to use the cross-entropy-loss with respect to the input of the encoder and the final output probabilities. During training the optimizer minimizes the Kullback-Leibler divergence of our output and target. The

Kullback-Leibler divergence is used for comparing two probabilistic distributions and defined by

$$KL(P||Q) = \sum_{i=1}^n \sum_{j=1}^m p_{ij} \log \frac{p_{ij}}{q_{ij}}$$

assuming that $P, Q \in \mathbb{R}^{n \times m}$ are stochastic matrices. At this point we know, that the output probabilities are forming a stochastic matrix, but our target (the input of the encoder) is a raw text file. We are able to define a measure on their similarities by taking the tokenized version of the input and apply a softmax function. Considering we have a known target matrix P we plug an unknown matrix into the equation and taking the derivative with respect to the plugged in variables. By taking the partial derivatives with respect to the output of the model. That way we can calculate the gradient, fortunately Python does it is automatic in Python,

7 Appendix

7.1 The Architecture of the Three Different Attention Mechanisms

8 References

- [1] Yusuxke Shibata, Takuya Kida, Shuichi Fukamachi, Masayuki Takeda, Ayumi Shinohara, Takeshi Shinohara, and Setsuo Arikawa. Byte pair encoding: A text compression scheme that accelerates pattern matching. 1999.
- [2] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [3] Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, Mohammad Norouzi, Wolfgang Macherey, Maxim Krikun, Yuan Cao, Qin Gao, Klaus Macherey, et al. Google’s neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.